计算机网络第一次实验

邹佳衡（2312322）

实验介绍

实验使用流式socket，设计一个聊天协议，要求聊天信息带有时间标签，请完整的说明监护消息的类型、语法、语义、时序等具体的消息处理方式。对聊天程序进行设计，给出模块划分说明、模块功能和模块的流程图。

在Windows系统下，利用C/C++对设计的程序进行实现。程序界面可以使用命令行的方式，但需要给出使用方法。编写程序时，只能使用基本的socket函数，不允许使用包装后的类或架构。

完成的程序应能支持中文和英文聊天。

采用多线程，支持多人聊天。

编写的程序应结构清晰，具有良好的可读性。

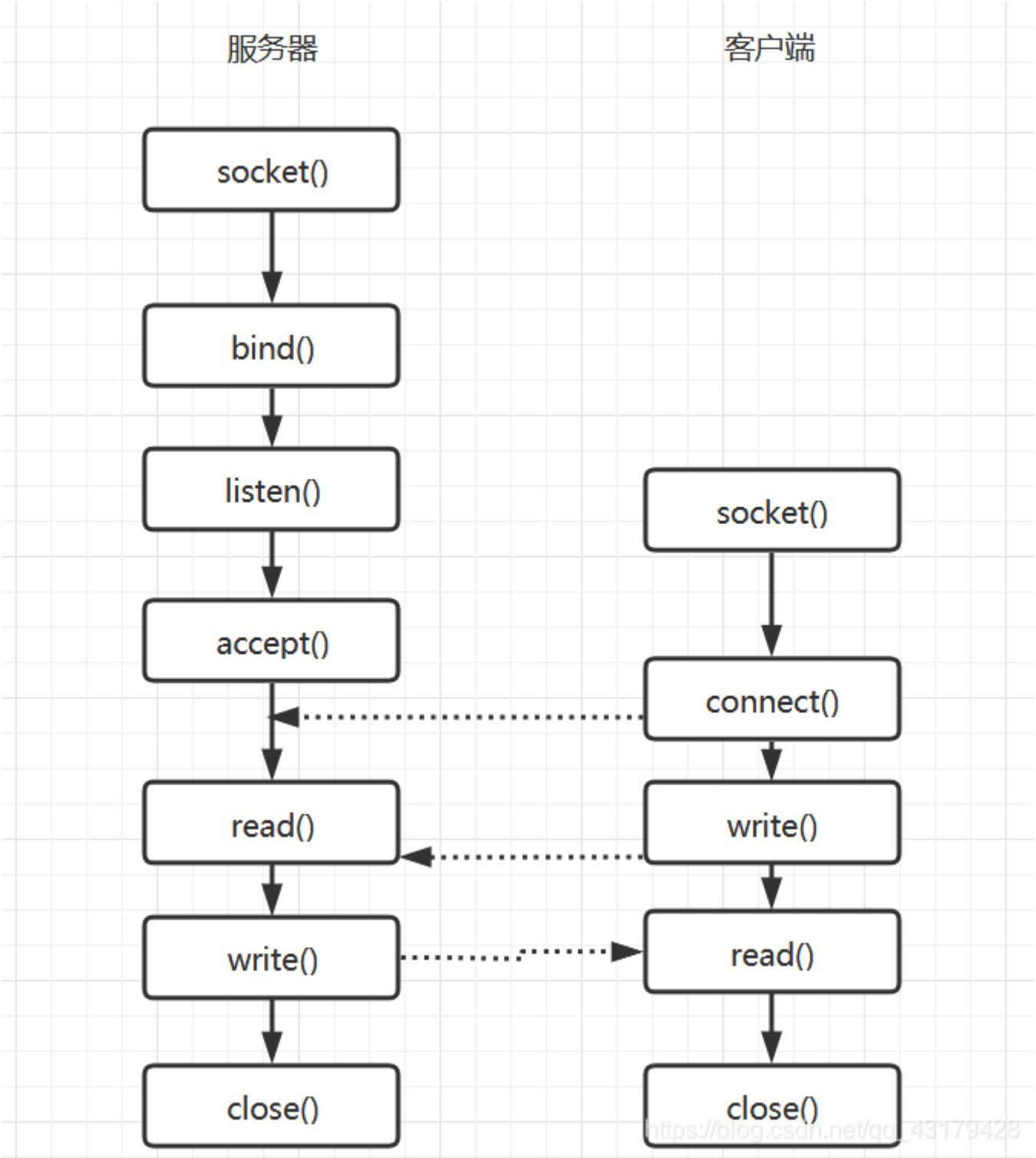
观察是否有数据包的丢失。

流式Socket

流式套接字是基于TCP协议的面向连接的套接字类型，提供可靠的、双向的、基于字节流的通信通道。这也就意味着它拥有以下特征：

- 面向连接：通讯前需要经历三次握手建立连接；
- 可靠性：数据无差错、不丢失、不重复、按序到达；
- 字节流导向：无消息边界，数据像水流一样连续；
- 全双工：双向通信，可同时收发数据

由于流式套接字是基于字节流进行信息传输的，因此在接收方看来，自己接收到的信息就是一串字节，而不是一条消息，如果是理想状况下的话，一次发送的消息可以直接接收，但是也有可能出现一条消息被分割成多个部分或者多条消息首尾相连，因此在编程时需要注意设置分割方法。



如图所示，流式套接字中主要使用以下函数进行建立连接和传递消息：

- socket(): 创建通讯端点, 指定协议和套接字类型
- bind(): 将套接字与本地IP和端口号绑定
- listen(): 将socket至于监听状态等待客户端连接
- accept(): 阻塞并等待客户端连接请求
- read(): 从客户端/服务端读取消息
- write(): 向客户端/服务端发送消息
- close(): 关闭Socket释放资源
- connect(): 向指定服务器的IP和端口号发起连接请求

协议设计

我们设计了一个简单但实用的基于TCP的聊天协议，下面从各个方面详细介绍：

协议架构

- 传输层: 基于TCP协议提供可靠的数据传输
- 通信模式: 服务器-客户端模式, 服务器负责消息转发和状态管理
- 连接方式: 长连接, 客户端连接后保持在线状态

消息类型

我们的消息分为客户端消息和服务端消息, 客户端消息包括昵称注册消息和普通聊天消息, 服务端消息包括系统通知消息和聊天消息广播

昵称注册消息是在客户端连接至服务端后, 会有系统提示用户为自己起一个昵称, 以纯文本字符串发送给服务端, 将昵称, 客户IP和Socket联系在一起, 为后来的聊天消息和消息广播提供便利. 除此之外, 我对昵称长度进行了限制, 同时考虑到了如果客户输入空昵称时用时间戳代替.

普通聊天消息是客户为自己取完昵称后, 用户输入的聊天内容, 以文本字符串发送给服务端, 以\r\n为分隔符, 同时当用户输入/quit时会退出聊天室.

系统通知消息是系统发布的提示和欢迎作用的消息, 用"==="包裹.

聊天消息广播是当客户发送普通聊天消息时, 服务端会将它广播到所有客户端, 消息会以"<时间 昵称> 消息"的格式进行输出.

使用流程

连接建立流程:

1. 客户端创建socket并连接到服务器
2. 服务器接受连接，设置socket为非阻塞模式
3. 服务器将新客户端添加到clients集合
4. 服务器发送欢迎消息给新客户端
5. 服务器广播用户加入通知给所有客户端
6. 客户端发送昵称作为第一条消息

消息发送流程:

1. 客户端输入消息并发送给服务器
2. 服务器接收消息
    - 如果是第一条消息，识别为昵称并存储
    - 如果是普通消息，添加时间戳并广播
    - 如果是/quit命令，关闭连接并广播离开通知

断开连接流程:

1. 客户端发送/quit命令或直接关闭连接
2. 服务器检测到连接断开
3. 服务器从clients、clientNames和clientIps中移除客户端信息
4. 服务器广播用户离开通知
5. 服务器关闭socket

可能的问题

- 缺少消息类型字段和长度字段的明确定义
- 无消息验证和加密机制
- 无昵称唯一性检查
- 无专门的错误消息格式

代码设计

这是一个基于C++/WinSock2实现的多线程聊天室程序, 包括一个服务器端 (server.cpp) 和一个客户端 (client.cpp). 可以支持多人在线聊天, 界面通过命令行的形式实现.

实现的数据结构

```
unordered_map<string, string> clientNames;  // socket句柄 -> 昵称
unordered_map<string, string> clientIps;    // socket句柄 -> IP地址
vector<SOCKET> clients;                     // 所有客户端socket
mutex clients_mutex;                        // 保护clients向量
```

这里主要需要介绍的是mutex clients_mutex, 这是一个互斥锁, 在多线程环境下有着重要的作用.

在服务端,我们有多个线程需要共享同样的数据, 比如clientsNames, clientsIps, client. 互斥锁可以保证同一时间只有一个线程可以访问这些资源, 比如在acceptNewClient函数中, 就使用了互斥锁来控制数据访问.

```
{
    std::lock_guard<std::mutex> lock(clients_mutex);
    clients.insert(client_socket);
    clientIps[std::to_string(client_socket)] = client_ip;
}
```

核心函数

```
void broadcastMessage(const vector<SOCKET>& clients, const char* message, SOCKET
sender) {
    string formatted_msg = string(message);
    if (formatted_msg.back() != '\n') {
        formatted_msg += "\r\n";
    }

    // 如果消息不是系统广播（如用户加入消息），则添加发送者昵称
    bool is_system_msg = formatted_msg.find("has joined the chat room!") !=
string::npos ||
        formatted_msg.find("has left the chat room!") != string::npos;

    if (!is_system_msg && sender != INVALID_SOCKET) {
        string sender_str = to_string(sender);
        string sender_name = "Anonymous";
```

```
        // 查找发送者昵称
        auto it = clientNames.find(sender_str);
        if (it != clientNames.end()) {
            sender_name = it->second;
        }

        // 查找当前时间
        auto now = chrono::system_clock::now();

        // 直接获取格式化的时间字符串
        auto t = chrono::system_clock::to_time_t(now);
        tm tm_time;
        localtime_s(&tm_time, &t);
        char buf[32];
        strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &tm_time);
        string time_str(buf);


        formatted_msg = "<" + time_str + " " + sender_name + "> " + formatted_msg;
    }

    for (SOCKET client : clients) {
        if (client != sender) {
            if (send(client, formatted_msg.c_str(), formatted_msg.length(), 0) ==
SOCKET_ERROR) {
                cout << "Failed to send: " << WSAGetLastError() << endl;
            }
        }
    }
}
```

该函数的主要功能是向发送者以外的所有客户端发送消息,在它的三个参数中, 第一个clients一般是客户端socket集合, message是要向客户端发送的字符串信息, sender是发送者的socket, 如果是系统消息就为INVALID_SOCKET. 可能是系统消息也可能是聊天消息, 通过检索关键字来判断两者, 在完成消息格式化后发送向所有客户端, 同时能捕获一些简单的错误并发出错误报告.

```
void acceptNewClient(SOCKET server_socket, vector<SOCKET>& clients, mutex&
clients_mutex) {
    sockaddr_in client_addr;
    int client_addr_len = sizeof(client_addr);

    SOCKET client_socket = accept(server_socket, (sockaddr*)&client_addr,
&client_addr_len);
    if (client_socket == INVALID_SOCKET) {
        cout << "accept failed: " << WSAGetLastError() << endl;
        return;
    }

    // 获取客户端IP地址
    char client_ip[INET_ADDRSTRLEN];
```

```
    inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, INET_ADDRSTRLEN);

    // 设置客户端socket为非阻塞
    u_long mode = 1; // 1表示非阻塞
    ioctlsocket(client_socket, FIONBIO, &mode);

    {
        lock_guard<mutex> lock(clients_mutex);
        clients.push_back(client_socket);
    }

    // 存储客户端IP地址
    string socket_str = to_string(client_socket);
    clientIps[socket_str] = client_ip;

    cout << "New client connection: " << client_ip << ":" <<
ntohs(client_addr.sin_port)
        << ", the total number of clients: " << clients.size() << endl;

    // 发送客户端欢迎消息
    string tips = "===Welcome to the chat room. Current number of online users: "
+ to_string(clients.size()) + "===\r\n";
    send(client_socket, tips.c_str(), tips.length(), 0);
}
```

这个函数的功能是接收新的连接, 设置socket为非阻塞模式, 记录客户端IP和端口, 发送欢迎消息. 其三个参数中, server_socket是服务器套接字, 用于接收新连接, clients是客户端集合, clients_mutex是互斥锁, 用于保护共享数据.

函数首先使用accept函数处理客户端连接请求, client_addr用于储存客户端地址信息.

接下来通过ioctlsocket()函数将客户端套接字设置为非阻塞模式, 这样在后续的recv()和send()操作中不会阻塞主线程. 非阻塞模式的优势在于其高并发性, 良好的响应能力和高资源利用效率. 适合需要处理多用户链接的场景.

这里简单介绍一下阻塞和非阻塞模式, 阻塞模式是套接字的默认工作模式. 在阻塞模式下, 当调用recv()、send()、accept()等IO操作时, 调用线程会一直等待直到操作完成或出错. 如果没有数据可读, 无法立即发送数据或没有新连接到达, 线程会被挂起, 不会占用CPU资源. 非阻塞模式下, IO操作会立即返回, 无论操作是否完成. 如果操作可以立即完成(如数据已准备好), 则返回操作结果; 如果操作不能立即完成, 则返回特定的错误码, 表示需要稍后重试.

在完成套接字设置后, 本线程对共享数据进行更新, 随后发送相关欢迎信息.

```
void handleClientMessages(vector<SOCKET>& clients, mutex& clients_mutex) {
    for (auto it = clients.begin(); it != clients.end(); ) {
        SOCKET client_socket = *it;
        char buffer[1024];
        string socket_str = to_string(client_socket);

        // 检查是否有数据
        fd_set readfds;
        FD_ZERO(&readfds);
```

```
        FD_SET(client_socket, &readfds);
        timeval timeout = { 0, 0 }; // 立即返回

        int activity = select(0, &readfds, nullptr, nullptr, &timeout);
        if (activity > 0 && FD_ISSET(client_socket, &readfds)) {
            int bytes_received = recv(client_socket, buffer, sizeof(buffer) - 1,
0);

            if (bytes_received > 0) {
                // 成功接收数据
                buffer[bytes_received] = '\0';
                string data(buffer);

                // 检查是否是昵称（用户的第一条消息）
                if (clientNames.find(socket_str) == clientNames.end()) {
                    // 这是昵称消息
                    string nickname = data;

                    // 存储昵称信息
                    clientNames[socket_str] = nickname;

                    // 获取客户端IP
                    string client_ip = clientIps[socket_str];
                    cout << "Client " << client_ip << " registered with nickname:
" << nickname << endl;

                    // 广播新用户加入消息
                    string join_msg = "==="+nickname + " has joined the chat
room!===\r\n";
                    lock_guard<mutex> lock(clients_mutex);
                    broadcastMessage(clients, join_msg.c_str(), client_socket);
                } else {
                    // 这是普通聊天消息
                    cout << "Received the message from " <<
clientNames[socket_str] << ": " << buffer;

                    // 广播消息给其他客户端
                    lock_guard<mutex> lock(clients_mutex);
                    broadcastMessage(clients, buffer, client_socket);
                }
                ++it;
            }
            else {
                // 客户端关闭连接
                string leaving_nickname = "Anonymous";

                // 获取离开的用户昵称
                auto it_name = clientNames.find(socket_str);
                if (it_name != clientNames.end()) {
                    leaving_nickname = it_name->second;
                    clientNames.erase(it_name);  // 从昵称映射中移除
                }

                // 移除IP信息
```

```
                clientIps.erase(socket_str);

                cout << "The client " << leaving_nickname << " is disconnected" <<
endl;

                // 先计算移除后的在线人数（减1，因为当前客户端还未移除）
                int online_count = clients.size() - 1;

                // 广播用户离开消息和在线人数
                string leave_msg = "==="+leaving_nickname + " has left the chat
room!" + "Current number of online users: " + to_string(online_count) +
"==="+"\r\n";
                lock_guard<mutex> lock(clients_mutex);
                broadcastMessage(clients, leave_msg.c_str(), INVALID_SOCKET);

                closesocket(client_socket);
                it = clients.erase(it);
            }
        }
        else {
            ++it;
        }
    }
}
```

handleClientMessages负责处理所有连接客户端的消息交互, 包括接收消息, 处理昵称注册, 广播聊天消息以及处理客户端断开连接的情况. 其有两个参数, clients是客户端集合, clients_mutex是互斥锁.

函数不断遍历客户端集合, 使用select()函数和非零超时（立即返回）来检查每个客户端是否有数据可读, 这是非阻塞IO的典型用法. FD_ZERO()清空文件描述符集合, FD_SET()将当前客户端套接字添加到集合中.

使用recv函数接收来自客户端的消息, 对昵称注册消息和普通聊天消息进行不同的处理.

当 bytes_received 为0（客户端正常关闭连接）或负数（发生错误）时: 获取离开用户的昵称, 如果未设置则使用"Anonymous", 从clientNames和clientIps映射中移除客户端信息, 计算更新后的在线用户数量, 广播用户离开消息给所有其他客户端, 使用INVALID_SOCKET表示这是系统消息, 关闭客户端套接字, 从clients向量中擦除该客户端, 并将迭代器指向下一个元素.

多线程实现

服务端多线程实现：

服务器端并未使用传统的多线程并发模型，而是采用了单线程 + IO多路复用的事件驱动模型。这种设计在Windows平台上通过select()函数实现。select函数本身可以同时监督多个客户端。

```
    while (running) {
        // 监听服务器socket和所有客户端socket
        fd_set readfds;
        FD_ZERO(&readfds);
        FD_SET(server_socket, &readfds);
```

```
        SOCKET max_fd = server_socket;

        // 添加所有客户端socket到集合
        {
            lock_guard<mutex> lock(clients_mutex);
            for (SOCKET client : clients) {
                FD_SET(client, &readfds);
                if (client > max_fd) max_fd = client;
            }
        }

        // 设置超时
        timeval timeout;
        timeout.tv_sec = 1;
        timeout.tv_usec = 0;

        // 等待活动
        int activity = select(0, &readfds, nullptr, nullptr, &timeout);

        if (activity == SOCKET_ERROR) {
            cout << "select error: " << WSAGetLastError() << endl;
            break;
        }

        if (activity > 0) {
            // 处理新连接
            if (FD_ISSET(server_socket, &readfds)) {
                acceptNewClient(server_socket, clients, clients_mutex);
            }

            // 处理客户端消息
            handleClientMessages(clients, clients_mutex);
        }
    }
```

以上是服务端的主循环，可以看到它先创建了一个文件描述符集合readfds并将服务端socket和所有客户端socket添加到集合中，设置超时时长为一秒，随后使用select监督有IO事件的文件描述符数量，如果是服务端socket有新IO事件就说明有新的客户端连接，否则就去处理客户端消息。

客户端明确使用了多线程模型，采用双线程设计：

主线程：处理用户输入和发送消息

```
    string message;
    while (connected) {
        getline(cin, message);

        if (!connected) break;

        if (message == "/quit") {
            connected = false;
```

```
            break;
        }

        // 发送消息到服务器
        message += "\r\n";

        if (send(client_socket, message.c_str(), message.length(), 0) ==
SOCKET_ERROR) {
            cout << "Failed to send: " << WSAGetLastError() << endl;
            connected = false;
            break;
        }
    }
```

接收线程：专门负责接收服务器消息并显示

```
    auto receiveFunc = [&]() {
        char buffer[1024];
        while (connected) {
            // 使用select检查是否有数据可读，避免阻塞
            fd_set readfds;
            FD_ZERO(&readfds);
            FD_SET(client_socket, &readfds);
            timeval timeout = { 0, 100000 }; // 100毫秒超时

            int activity = select(0, &readfds, nullptr, nullptr, &timeout);

            if (activity > 0 && FD_ISSET(client_socket, &readfds)) {
                int bytes_received = recv(client_socket, buffer, sizeof(buffer) -
1, 0);

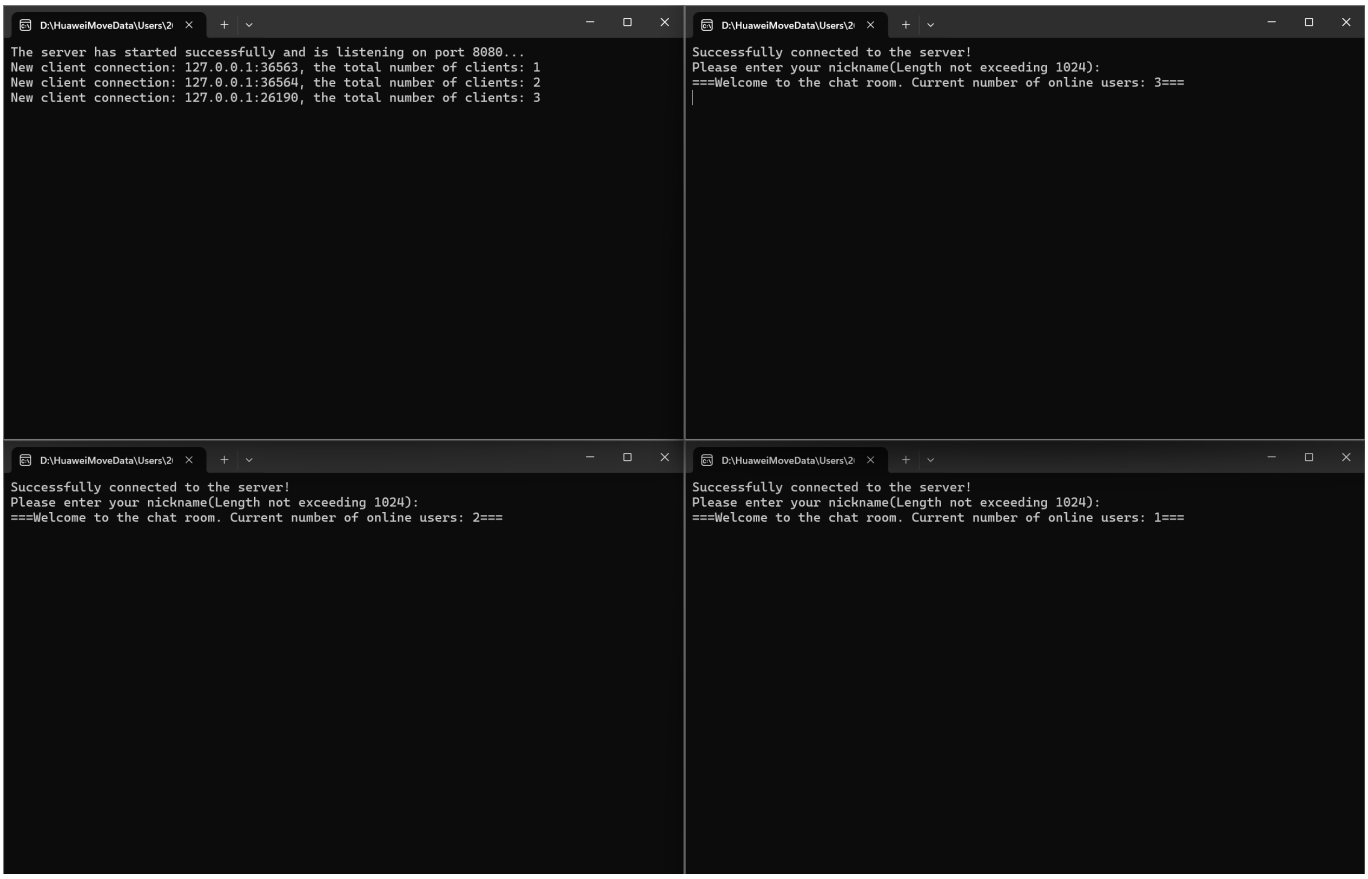
                if (bytes_received > 0) {
                    buffer[bytes_received] = '\0';
                    cout << buffer;
                }
                else if (bytes_received == 0) {
                    cout << "\nThe server is disconnected." << endl;
                    connected = false;
                    break;
                }
                else {
                    // 非阻塞模式下，WSAEWOULDBLOCK是正常的，不是错误
                    int error_code = WSAGetLastError();
                    if (error_code != WSAEWOULDBLOCK) {
                        cout << "\nReception error: " << error_code << endl;
                        connected = false;
                        break;
                    }
                }
            }
            // 如果select超时，继续循环检查connected标志
        }
```

```
    };

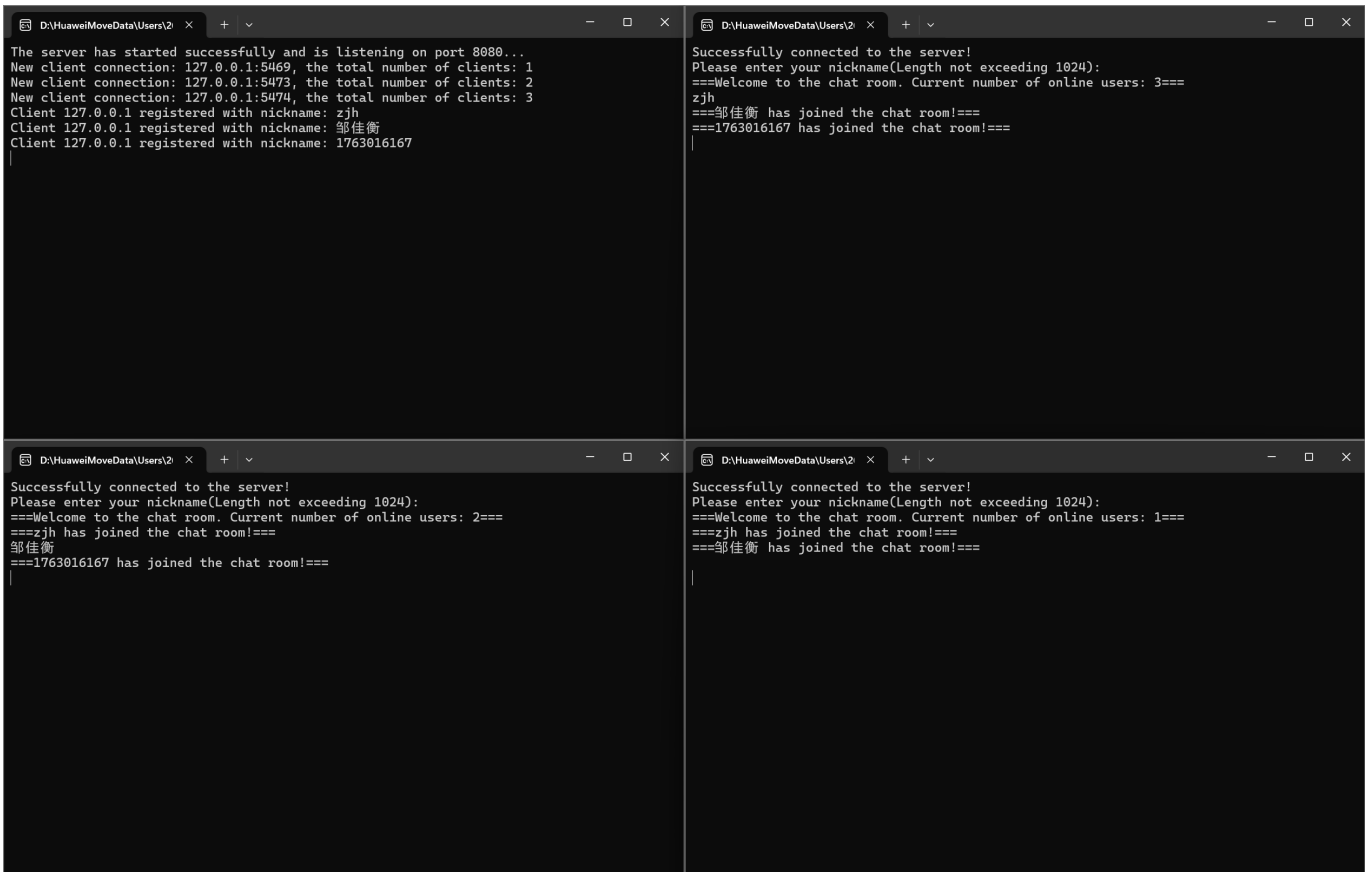
    // 启动接收线程
    thread receiver(receiveFunc);
```

在聊天客户端代码中，connected原子布尔变量是一个核心的线程间通信机制，用于协调主线程和接收线程的运行状态。接收线程在while (connected)循环中运行，只要connected为true，就会持续检查并接收服务器消息。主要是为了实现两个线程的状态同步，只要一个线程中connected为false，两个线程都会退出。

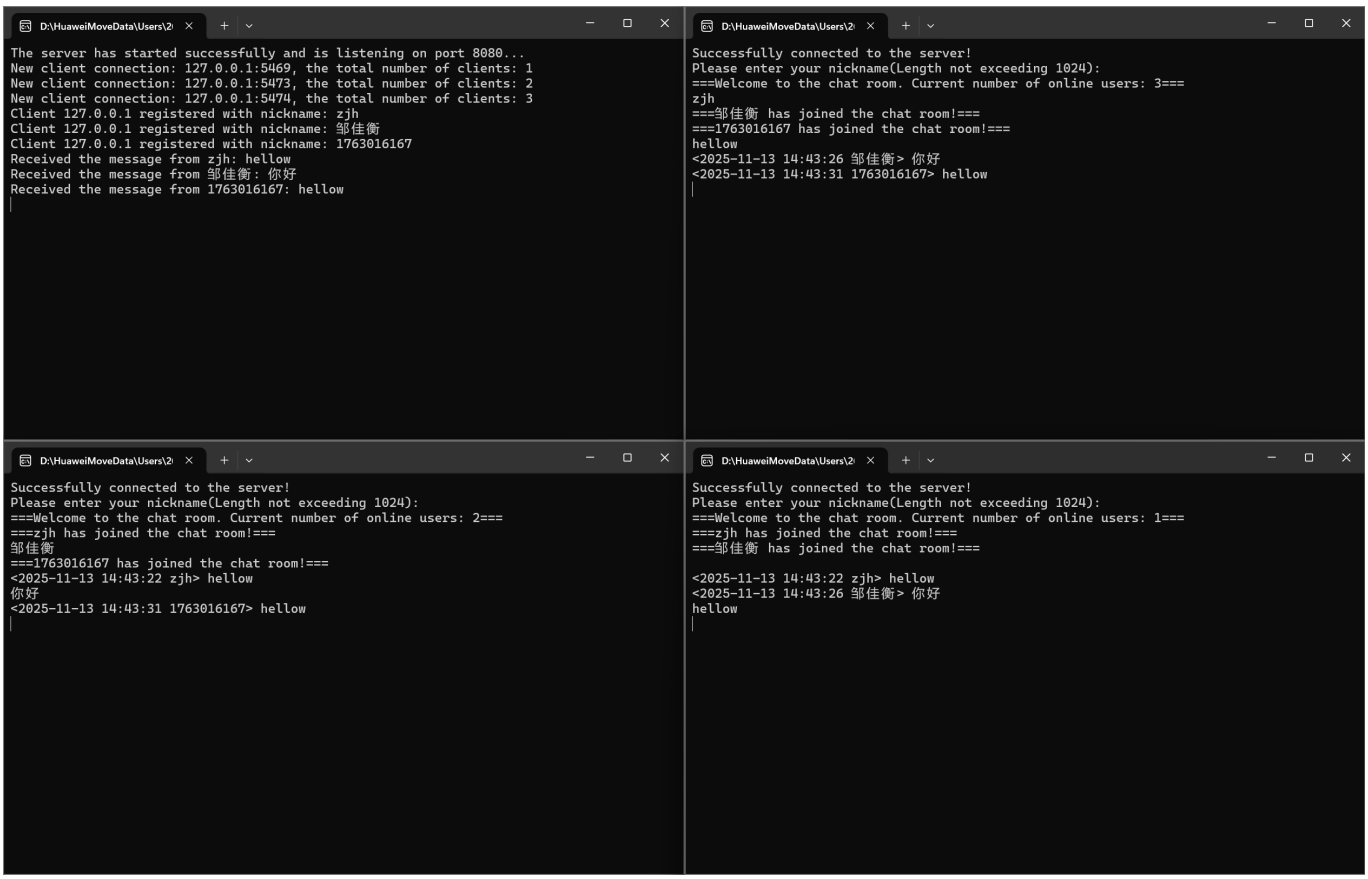
实现效果



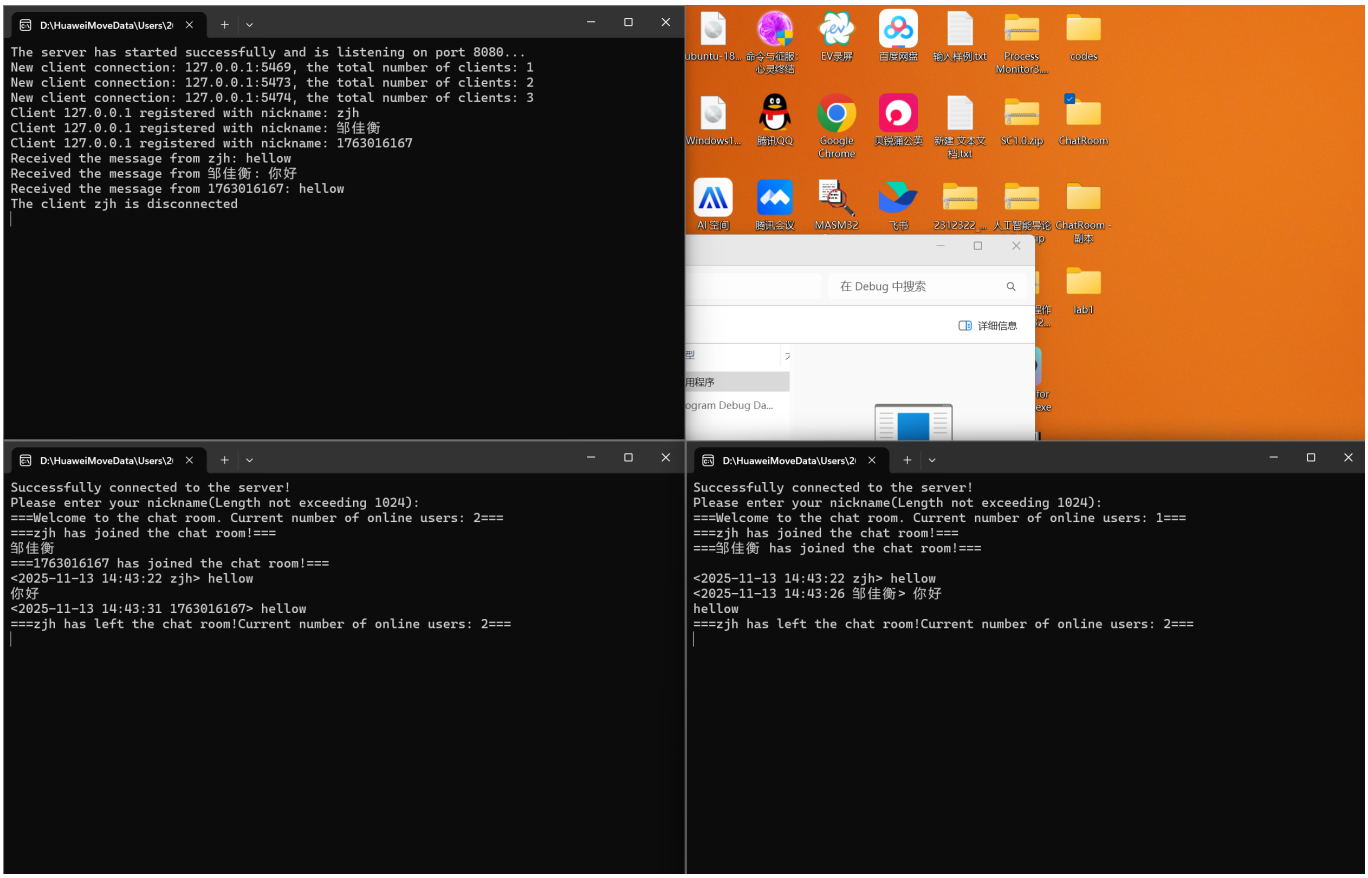
我们打开一个服务端和三个客户端，它们能够正常连接。



三个用户分别为自己起名zjh、邹佳衡和空名字，空名字以起名瞬间的时间戳代替。



能够支持中英文实时聊天



zjh用户输入/quit正常退出，能够实现正常退出功能。直接点叉也能退出，唯一的区别是服务端会弹出异常掉线提示，其他功能均正常。